5

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Compose ile Çoklu Servis: Spring Boot + PostgreSQL Çiftliği

Multi-Service Stacks with Docker Compose: Spring Boot + PostgreSQL

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Restoranda tek başına yemek yapmak kolaydır. Ama aynı anda ekmek fırınını, et ızgarasını ve tatlı reyonunu senkronize başlatmak için bir şefe ve nota kağıdına ihtiyacınız vardır. Docker Compose o nota kağıdıdır (`docker-compose.yml`). Tek bir tuşla (`docker compose up`) tüm orkestra kusursuz sırada sahneye çıkar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Playing a solo flute is easy. Conducting a 20-piece symphony requires sheet music! Docker Compose is the master sheet music (`docker-compose.yml`). With a single command (`docker compose up`), your database, Redis cache, and Spring Boot backend spawn in perfect harmony!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker Compose bildirici (declarative) bir YAML dosyasıyla servisleri, ağları ve diskleri tek bir çatı altında tanımlar. `depends_on` mekanizması başlatma sırasını belirler; `healthcheck` bloğu ile birleştirildiğinde Spring Boot servisinin PostgreSQL tamamen 'ready' olana kadar beklemesi sağlanabilir. Ortam değişkenleri (`environment`), port haritalamaları (`ports`) ve secret dosyaları tek merkezden yönetilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Docker Compose declaratively defines multi-container applications. Using `depends_on` with `condition: service_healthy`, Compose ensures your Spring Boot service pauses booting until PostgreSQL is genuinely accepting TCP connections, eliminating startup connection crashes.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# docker-compose.yml
services:
  postgres-db:
    image: postgres:16-alpine
    container_name: postgres-db
    environment:
      POSTGRES_DB: coffeedb
      POSTGRES_USER: coffeeuser
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U coffeeuser -d coffeedb"]
      interval: 5s
      timeout: 5s
      retries: 5

  coffee-app:
    build: .
    container_name: coffee-app
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-db:5432/coffeedb
      SPRING_DATASOURCE_USERNAME: coffeeuser
      SPRING_DATASOURCE_PASSWORD: secretpassword
    depends_on:
      postgres-db:
        condition: service_healthy

volumes:
  postgres-data:
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`depends_on` tek başına sadece veritabanı konteynerinin başladığını doğrular; veritabanı motorunun bağlantı kabul etmeye hazır olduğunu garanti etmez. Mutlaka `condition: service_healthy` ile birlikte kullanılmalıdır!

🇬🇧 WATCH OUT

A bare `depends_on` only waits for the container process to spawn, NOT for PostgreSQL to be ready! Always pair it with healthcheck conditions.

🎯 Özet Çıkarımlar

• Tek bir komut (`docker compose up -d`) ile tüm mikroservis yığını ayağa kalkar.

• Healthcheck blokları servisler arası yarış durumlarını (race condition) önler.

• Yerel geliştirme ortamını üretim ortamına en yakın hale getirir.

🎯 Key Takeaways

• Spawn entire multi-tier architectures with a single `docker compose up -d` command.

• Healthchecks eliminate startup race conditions between apps and databases.

• Mirrors production topology on local developer machines seamlessly.